Lab 2:

IDS code:

```
def ids(start, goal, max_depth=50):

    def dls(state, depth, path, visited):
        if state == goal:
            return path + [state]

        if depth == 0:
            return None

        zero = state.index(0)
        row, col = zero // 3, zero % 3

        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for dr, dc in moves:
            nr, nc = row + dr, col + dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                pos = nr * 3 + nc
                new_state = state[:]
                new_state[zero], new_state[pos] = new_state[pos],
new_state[zero]

                key = tuple(new_state)

                if key not in visited:
                    visited.add(key)
                    result = dls(
                        new_state,
                        depth - 1,
                        path + [state],
                        visited
                    )

                    if result is not None:
                        return result

                    visited.remove(key)

        return None

    for limit in range(max_depth + 1):
        result = dls(start, limit, [], {tuple(start)})

        if result is not None:
```

```
            return result

    return None


start = [5, 4, 6, 1, 8, 7, 3, 2, 0]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

result = ids(start, goal)

if result:
    print("Solution found:")
    for state in result:
        for i in range(0, 9, 3):
            print(state[i:i + 3])
        print()
else:
    print("No solution found within the depth limit")
```

output:

```
Solution found:
[5, 4, 6]
[1, 8, 7]
[3, 2, 0]

[5, 4, 6]
[1, 8, 0]
[3, 2, 7]

[5, 4, 6]
[1, 0, 8]
[3, 2, 7]

[5, 4, 6]
[1, 2, 8]
[3, 0, 7]

[5, 4, 6]
[1, 2, 8]
[3, 7, 0]

[5, 4, 6]
[1, 2, 0]
[3, 7, 8]

[5, 4, 6]
[1, 0, 2]
[3, 7, 8]

[5, 4, 6]
[0, 1, 2]
[3, 7, 8]
```

```
[5, 1, 2]
[3, 7, 8]

[4, 0, 6]
[5, 1, 2]
[3, 7, 8]

[4, 1, 6]
[5, 0, 2]
[3, 7, 8]

[4, 1, 6]
[0, 5, 2]
[3, 7, 8]

[4, 1, 6]
[3, 5, 2]
[0, 7, 8]

[4, 1, 6]
[3, 5, 2]
[7, 0, 8]

[4, 1, 6]
[3, 0, 2]
[7, 5, 8]

[4, 1, 6]
[0, 3, 2]
[7, 5, 8]

[0, 1, 6]
[4, 3, 2]
[7, 5, 8]
```

```
    [7, 5, 8]

...  [1, 0, 6]
    [4, 3, 2]
    [7, 5, 8]

    [1, 3, 6]
    [4, 0, 2]
    [7, 5, 8]

    [1, 3, 6]
    [4, 2, 0]
    [7, 5, 8]

    [1, 3, 0]
    [4, 2, 6]
    [7, 5, 8]

    [1, 0, 3]
    [4, 2, 6]
    [7, 5, 8]

    [1, 2, 3]
    [4, 0, 6]
    [7, 5, 8]

    [1, 2, 3]
    [4, 5, 6]
    [7, 0, 8]

    [1, 2, 3]
    [4, 5, 6]
    [7, 8, 0]
```

2. DFS code puzzle

```
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def dfs(state, depth, limit, path):
    if state == goal:
        return path

    if depth == limit:
        return None
```

```
    zero = state.index(0)
    row, col = zero // 3, zero % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)

            new_state[zero], new_state[pos] = (
                new_state[pos], new_state[zero]
            )

            new_state = tuple(new_state)

            if new_state not in path:
                result = dfs(
                    new_state, depth + 1, limit,
                    path + [new_state]
                )

                if result:
                    return result

    return None


start = tuple(map(int, input("Enter 9 numbers: ").split()))
limit = int(input("Enter depth limit: "))

solution = dfs(start, 0, limit, [start])

if solution:
    print("Solution found:")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i:i+3])
        print()
else:
    print("Solution not found within depth limit.")
```

output:

```
    print("Solution not found within d
```

```
Enter 9 numbers: 1 2 3 4 5 6 7 8 0
Enter depth limit: 5
Solution found:
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```